

Data Structures

3. Sets

Sets are unordered collections of unique elements. The sets are unalterable, not indexed.

To create a set, Python provides the `set()` function. One or more elements constituting the set are provided, separated by commas and surrounded by braces (`{}`):

```
new_set = set({"Alger", "Annaba", "Oran", "Biskra"})
print(new_set)
```

Output: {'Biskra', 'Oran', 'Alger', 'Annaba'}

Equivalently, rather than using the `set()` function, the set can only be defined using the brackets:

```
new_set = {"Alger", "Annaba", "Oran", "Biskra"}
print(new_set)
```

Output: {'Biskra', 'Oran', 'Alger', 'Annaba'}

On the other hand, if the set is empty, Python returns an error if the `set()` function is not used: it is necessary to use the `set` function :

```
empty_set = {}
type(empty_set)
```

Output: <class 'dict'>

The type of the object we have just created is not set but dict (i.e. Section 1.4). Also, to create the empty set, we use `set()`:

```
empty_set = set()
print(type(empty_set))
```

Output: <class 'set'>

During the creation of a set, if there are duplicates in the values provided, these will be deleted to keep only one value:

```
new_set = set({"Alger", "Annaba", "Oran", "Biskra"})
print(new_set)
```

Output: {'Biskra', 'Oran', 'Alger', 'Annaba'}

The length of a set is obtained using the `len()` function:

```
print(len(new_set))
```

Output: 4

3.1. Modifications

3.1.1 Adding an Element

To add an element to a set, Python offers the add() method:

```
new_set.add("Sidi Bel Abbes")  
print(new_set)
```

Output: {'Biskra', 'Oran', 'Alger', 'Annaba', 'Sidi Bel Abbes'}

3.1.2. Deletin

To remove a value from a set, Python offers the method remove():

```
new_set.remove("Annaba")  
print(new_set)
```

Output: {'Biskra', 'Oran', 'Alger', 'Sidi Bel Abbes'}

If the value to be deleted is not in the set, Python returns an error message:

```
new_set.remove("Tlemcen")
```

Output:

```
## Error in py_call_impl(callable, dots$args, dots$keywords):  
KeyError: 'Tlemcen'  
##  
## Detailed traceback:  
## File "<string>", line 1, in <module>
```

To avoid this error, Python offers the discard() method, which does not return an error if the value to be deleted is not in the set:

```
new_set.discard("Tlemcen")  
print(new_set)
```

Output: {'Biskra', 'Oran', 'Alger', 'Sidi Bel Abbes'}

3.1.3. Deleting the Set

To delete a set, Python offers the del() function:

```
del new_set
```

3.2 Belonging test

One of the advantages of sets is the quick search for presence or absence of values (faster than in a list). As with the lists, the belonging tests are performed using the operator in:

```
print("Oran" in new_set)
```

Output: True

```
print("Tlemcen" in new_set)
```

Output: False

3.3. Operations

Sets can be combined using the operators `union()`, `intersection()`, `difference()` and `symmetric_difference()`:

```
set1 = {"Alger", "Annaba", "Oran", "Biskra"}
set2 = {"Oran", "Biskra", "Tlemcen", "Sidi Bel Abbes"}
print(set1.union(set2))
```

Output: {'Biskra', 'Alger', 'Sidi Bel Abbes', 'Tlemcen', 'Annaba', 'Oran'}

```
print(set1.intersection(set2))
```

Output: {'Biskra', 'Oran'}

```
print(set1.difference(set2))
```

Output: {'Alger', 'Annaba'}

```
print(set1.symmetric_difference(set2))
```

Output: {'Biskra', 'Alger', 'Sidi Bel Abbes', 'Tlemcen', 'Annaba'}

3.4. Iteration

Sets can be iterated on:

```
for city in set1:
    print(city)
```

Output:

```
Biskra
Oran
Alger
Annaba
```

3.5. Conversion

Sets can be converted to lists and vice versa:

```
list1 = list(set1)
print(list1)
```

Output: ['Biskra', 'Oran', 'Alger', 'Annaba']

```
set1 = set(list1)
print(set1)
```

Output: {'Biskra', 'Oran', 'Alger', 'Annaba'}

3.6. Copying a Set

To copy a set, as for lists (c.f. Section 1), the equality sign should not be used. Copying a set is done using the `copy()` method:

```
set1 = {"Alger", "Annaba", "Oran", "Biskra"}
set2 = set1.copy()
set2.add("Sidi Bel Abbes")
print("set2: ", set2)
```

Output: set2: {'Biskra', 'Oran', 'Alger', 'Sidi Bel Abbes', 'Annaba'}

```
print("set1: ", set1)
```

```
Output: set1: {'Biskra', 'Oran', 'Alger', 'Annaba'}
```

3.7. Comparison

Sets can be compared using the operators `==` and `!=`:

```
set1 = {"Alger", "Annaba", "Oran", "Biskra"}
set2 = {"Oran", "Biskra", "Tlemcen", "Sidi Bel Abbes"}
set3 = set1.copy()
print(set1 == set2)
```

```
Output: False
```

```
print(set1 == set3)
```

```
Output: True
```

3.8. Frozen Sets

A frozen set is a set that cannot be modified. It is created using the `frozenset()` function:

```
frozen_set = frozenset({"Alger", "Annaba", "Oran", "Biskra"})
print(frozen_set)
```

```
Output: frozenset({'Biskra', 'Oran', 'Alger', 'Annaba'})
```

The main difference with a set is that a frozen set cannot be modified:

```
frozen_set.add("Sidi Bel Abbes")
```

```
Output:
```

```
## Error in py_call_impl(callable, dots$args, dots$keywords):
AttributeError: 'frozenset' object has no attribute 'add'
##
## Detailed traceback:
##   File "<string>", line 1, in <module>
```

3.9. Sets of Sets

Sets can contain other sets:

```
set1 = frozenset({"Alger", "Annaba", "Oran", "Biskra"})
set2 = frozenset({"Oran", "Biskra", "Tlemcen", "Sidi Bel Abbes"})
set3 = {set1, set2}
print(set3)
```

```
Output: {frozenset({'Biskra', 'Oran', 'Alger', 'Annaba'}),
frozenset({'Biskra', 'Oran', 'Sidi Bel Abbes', 'Tlemcen'})}
```